xref: /src

Up to higher level directory

Name	Date	Size
bin/	Today	
build.sh	13-May-2025	77.8K
BUILDING	26-Apr-2024	41.8K
common/	09-Nov-2022	
compat/	Today	
dist/	Today	
distrib/	Today	
doc/	19-May-2025	
etc/	Today	
games/	Today	
include/	Today	
lib/	Today	
libexec/	Today	
Makefile	05-May-2025	16K
Makefile.inc	02-May-2018	355
README.md	05-Sep-2021	1.7K
regress/	Today	
rescue/	09-Nov-2022	
sbin/	Today	
share/	Today	
sys/	Today	
tests/	Today	
tools/	Today	
UPDATING	28-Apr-2025	21.6K
usr.bin/	Today	
usr.sbin/	Today	

README.md

```
NetBSD
======

NetBSD is a free, fast, secure, and highly portable Unix-like Open
Source operating system.  It is available for a [wide range of
platforms](https://wiki.NetBSD.org/ports/), from large-scale servers
and powerful desktop systems to handheld and embedded devices.

Building
--------

You can cross-build NetBSD from most UNIX-like operating systems.
To build for amd64 (x86_64), in the src directory:

    ./build.sh -U -u -j4 -m amd64 -O ~/obj release

Additional build information available in the [BUILDING](BUILDING) file.

Binaries
--------

- [Daily builds](https://nycdn.NetBSD.org/pub/NetBSD-daily/HEAD/latest/)
- [Releases](https://cdn.NetBSD.org/pub/NetBSD/)

Testing
-------
```

On a running NetBSD system:

```
cd /usr/tests; atf-run | atf-report
```

Troubleshooting

- Send bugs and patches [via web form](https://www.NetBSD.org/cgi-bin/sendpr.cgi?gndb=netbsd).
- Subscribe to the [mailing lists](https://www.NetBSD.org/mailinglists/).
  The [netbsd-users](https://www.NetBSD.org/mailinglists/#netbsd-users) list is a good choice for many problems; watch [current-users](https://www.NetBSD.org/mailinglists/#current-users) if you follow the bleeding edge of NetBSD-current.
- Join the community IRC channel [#netbsd @ libera.chat](https://web.libera.chat/#netbsd).

Latest sources

To fetch the main CVS repository:

```
cvs -d anoncvs (a] anoncvs.NetBSD.org:/cvsroot checkout -P src
```

To work in the Git mirror, which is updated every few hours from CVS:

```
git clone https://github.com/NetBSD/src.git
```

Additional Links

- [The NetBSD Guide](https://www.NetBSD.org/docs/guide/en/)
- [NetBSD manual pages](https://man.NetBSD.org/)
- [NetBSD Cross-Reference](https://nxr.NetBSD.org/)

served by {OpenGrok on Java

Indexes created Tue May 20 18:09:42 GMT 2025

OpenGrok

Cross Reference: getopt_long.c

xref: /src/lib/libc/stdlib/getopt_long.c

getopt_long.c revision 1.15

```
/*	$NetBSD: getopt_long.c,v 1.15 2002/01/31 22:43:40 tv Exp $	*/

/*-
 * Copyright (c) 2000 The NetBSD Foundation, Inc.
 * All rights reserved.
 *
 * This code is derived from software contributed to The NetBSD Foundation
 * by Dieter Baron and Thomas Klausner.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. All advertising materials mentioning features or use of this software
 *    must display the following acknowledgement:
 *        This product includes software developed by the NetBSD
 *        Foundation, Inc. and its contributors.
 * 4. Neither the name of The NetBSD Foundation nor the names of its
 *    contributors may be used to endorse or promote products derived
 *    from this software without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE NETBSD FOUNDATION, INC. AND CONTRIBUTORS
 * ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED
 * TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
 * PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE FOUNDATION OR CONTRIBUTORS
 * BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
 * CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
 * SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
 * INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
 * CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
 * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
 * POSSIBILITY OF SUCH DAMAGE.
 */

#include <sys/cdefs.h>
#if defined(LIBC_SCCS) && !defined(lint)
__RCSID("$NetBSD: getopt_long.c,v 1.15 2002/01/31 22:43:40 tv Exp $");
```

```
42  |#endif /* LIBC_SCCS and not lint */
43  |
44  |#include "namespace.h"
45  |
46  |#include <assert.h>
47  |#include <err.h>
48  |#include <errno.h>
49  |#include <getopt.h>
50  |#include <stdlib.h>
51  |#include <string.h>
52  |
53  |#if HAVE_CONFIG_H && !HAVE_GETOPT_LONG && !HAVE_DECL_OPTIND
54  |#define REPLACE_GETOPT
55  |#endif
56  |
57  |#ifdef REPLACE_GETOPT
58  |#ifdef __weak_alias
59  |__weak_alias(getopt,_getopt)
60  |#endif
61  |int     opterr = 1;             /* if error message should be printed */
62  |int     optind = 1;             /* index into parent argv vector */
63  |int     optopt = '?';           /* character checked for validity */
64  |int     optreset;               /* reset getopt */
65  |char    *optarg;                /* argument associated with option */
66  |#elif HAVE_CONFIG_H && !HAVE_DECL_OPTRESET
67  |static int optreset;
68  |#endif
69  |
70  |#ifdef __weak_alias
71  |__weak_alias(getopt_long,_getopt_long)
72  |#endif
73  |
74  |#if !HAVE_GETOPT_LONG
75  |#define IGNORE_FIRST    (*options == '-' || *options == '+')
76  |#define PRINT_ERROR     ((opterr) && ((*options != ':') \
77  |                                    || (IGNORE_FIRST && options[1] != ':')))
78  |#define IS_POSIXLY_CORRECT (getenv("POSIXLY_CORRECT") != NULL)
79  |#define PERMUTE         (!IS_POSIXLY_CORRECT && !IGNORE_FIRST)
80  |/* XXX: GNU ignores PC if *options == '-' */
81  |#define IN_ORDER        (!IS_POSIXLY_CORRECT && *options == '-')
82  |
83  |/* return values */
84  |#define BADCH   (int)'?'
85  |#define BADARG          ((IGNORE_FIRST && options[1] == ':') \
86  |                         || (*options == ':') ? (int)':' : (int)'?')
87  |#define INORDER (int)1
88  |
89  |#define EMSG    ""
90  |
91  |static int getopt_internal __P((int, char * const *, const char *));
```

```
92  static int gcd __P((int, int));
93  static void permute_args __P((int, int, int, char * const *));
94
95  static char *place = EMSG; /* option letter processing */
96
97  /* XXX: set optreset to 1 rather than these two */
98  static int nonopt_start = -1; /* first non option argument (for permute) */
99  static int nonopt_end = -1;   /* first option after non options (for permute) */
100
101 /* Error messages */
102 static const char recargchar[] = "option requires an argument -- %c";
103 static const char recargstring[] = "option requires an argument -- %s";
104 static const char ambig[] = "ambiguous option -- %.*s";
105 static const char noarg[] = "option doesn't take an argument -- %.*s";
106 static const char illoptchar[] = "unknown option -- %c";
107 static const char illoptstring[] = "unknown option -- %s";
108
109
110 /*
111  * Compute the greatest common divisor of a and b.
112  */
113 static int
114 gcd(a, b)
115  int a;
116  int b;
117 {
118  int c;
119
120  c = a % b;
121  while (c != 0) {
122          a = b;
123          b = c;
124          c = a % b;
125  }
126
127  return b;
128 }
129
130 /*
131  * Exchange the block from nonopt_start to nonopt_end with the block
132  * from nonopt_end to opt_end (keeping the same order of arguments
133  * in each block).
134  */
135 static void
136 permute_args(panonopt_start, panonopt_end, opt_end, nargv)
137  int panonopt_start;
138  int panonopt_end;
139  int opt_end;
140  char * const *nargv;
141 {
```

```
142 | int cstart, cyclelen, i, j, ncycle, nnonopts, nopts, pos;
143 | char *swap;
144 |
145 | _DIAGASSERT(nargv != NULL);
146 |
147 | /*
148 |  * compute lengths of blocks and number and size of cycles
149 |  */
150 | nnonopts = panonopt_end - panonopt_start;
151 | nopts = opt_end - panonopt_end;
152 | ncycle = gcd(nnonopts, nopts);
153 | cyclelen = (opt_end - panonopt_start) / ncycle;
154 |
155 | for (i = 0; i < ncycle; i++) {
156 |         cstart = panonopt_end+i;
157 |         pos = cstart;
158 |         for (j = 0; j < cyclelen; j++) {
159 |                 if (pos >= panonopt_end)
160 |                         pos -= nnonopts;
161 |                 else
162 |                         pos += nopts;
163 |                 swap = nargv[pos];
164 |                 /* LINTED const cast */
165 |                 ((char **) nargv)[pos] = nargv[cstart];
166 |                 /* LINTED const cast */
167 |                 ((char **)nargv)[cstart] = swap;
168 |         }
169 | }
170 |}
171 |
172 |/*
173 | * getopt_internal --
174 | *       Parse argc/argv argument vector.  Called by user level routines.
175 | *  Returns -2 if -- is found (can be long option or end of options marker).
176 | */
177 |static int
178 |getopt_internal(nargc, nargv, options)
179 | int nargc;
180 | char * const *nargv;
181 | const char *options;
182 |{
183 | char *oli;                              /* option letter list index */
184 | int optchar;
185 |
186 | _DIAGASSERT(nargv != NULL);
187 | _DIAGASSERT(options != NULL);
188 |
189 | optarg = NULL;
190 |
191 | /*
```

```
192     * XXX Some programs (like rsyncd) expect to be able to
193     * XXX re-initialize optind to 0 and have getopt_long(3)
194     * XXX properly function again.  Work around this braindamage.
195     */
196   if (optind == 0)
197           optind = 1;
198
199   if (optreset)
200           nonopt_start = nonopt_end = -1;
201  start:
202   if (optreset || !*place) {              /* update scanning pointer */
203           optreset = 0;
204           if (optind >= nargc) {          /* end of argument vector */
205                   place = EMSG;
206                   if (nonopt_end != -1) {
207                           /* do permutation, if we have to */
208                           permute_args(nonopt_start, nonopt_end,
209                               optind, nargv);
210                           optind -= nonopt_end - nonopt_start;
211                   }
212                   else if (nonopt_start != -1) {
213                           /*
214                            * If we skipped non-options, set optind
215                            * to the first of them.
216                            */
217                           optind = nonopt_start;
218                   }
219                   nonopt_start = nonopt_end = -1;
220                   return -1;
221           }
222           if ((*(place = nargv[optind]) != '-')
223               || (place[1] == '\0')) {    /* found non-option */
224                   place = EMSG;
225                   if (IN_ORDER) {
226                           /*
227                            * GNU extension:
228                            * return non-option as argument to option 1
229                            */
230                           optarg = nargv[optind++];
231                           return INORDER;
232                   }
233                   if (!PERMUTE) {
234                           /*
235                            * if no permutation wanted, stop parsing
236                            * at first non-option
237                            */
238                           return -1;
239                   }
240                   /* do permutation */
```

```
241 |                 if (nonopt_start == -1)
242 |                         nonopt_start = optind;
243 |                 else if (nonopt_end != -1) {
244 |                         permute_args(nonopt_start, nonopt_end,
245 |                             optind, nargv);
246 |                         nonopt_start = optind -
247 |                             (nonopt_end - nonopt_start);
248 |                         nonopt_end = -1;
249 |                 }
250 |                 optind++;
251 |                 /* process next argument */
252 |                 goto start;
253 |         }
254 |         if (nonopt_start != -1 && nonopt_end == -1)
255 |                 nonopt_end = optind;
256 |         if (place[1] && *++place == '-') {      /* found "--" */
257 |                 place++;
258 |                 return -2;
259 |         }
260 | }
261 | if ((optchar = (int)*place++) == (int)':' ||
262 |     (oli = strchr(options + (IGNORE_FIRST ? 1 : 0), optchar)) == NULL) {
263 |         /* option letter unknown or ':' */
264 |         if (!*place)
265 |                 ++optind;
266 |         if (PRINT_ERROR)
267 |                 warnx(illoptchar, optchar);
268 |         optopt = optchar;
269 |         return BADCH;
270 | }
271 | if (optchar == 'W' && oli[1] == ';') {          /* -W long-option */
272 |         /* XXX: what if no long options provided (called by getopt)? */
273 |         if (*place)
274 |                 return -2;
275 |
276 |         if (++optind >= nargc) {        /* no arg */
277 |                 place = EMSG;
278 |                 if (PRINT_ERROR)
279 |                         warnx(recargchar, optchar);
280 |                 optopt = optchar;
281 |                 return BADARG;
282 |         } else                          /* white space */
283 |                 place = nargv[optind];
284 |         /*
285 |          * Handle -W arg the same as --arg (which causes getopt to
286 |          * stop parsing).
287 |          */
288 |         return -2;
289 | }
290 | if (*++oli != ':') {                    /* doesn't take argument */
```

```
291 |         if (!*place)
292 |                 ++optind;
293 | } else {                                /* takes (optional) argument */
294 |         optarg = NULL;
295 |         if (*place)                     /* no white space */
296 |                 optarg = place;
297 |         /* XXX: disable test for :: if PC? (GNU doesn't) */
298 |         else if (oli[1] != ':') {       /* arg not optional */
299 |                 if (++optind >= nargc) {        /* no arg */
300 |                         place = EMSG;
301 |                         if (PRINT_ERROR)
302 |                                 warnx(recargchar, optchar);
303 |                         optopt = optchar;
304 |                         return BADARG;
305 |                 } else
306 |                         optarg = nargv[optind];
307 |         }
308 |         place = EMSG;
309 |         ++optind;
310 | }
311 | /* dump back option letter */
312 | return optchar;
313 |}
314 |
315 |#ifdef REPLACE_GETOPT
316 |/*
317 | * getopt --
318 | *       Parse argc/argv argument vector.
319 | *
320 | * [eventually this will replace the real getopt]
321 | */
322 |int
323 |getopt(nargc, nargv, options)
324 | int nargc;
325 | char * const *nargv;
326 | const char *options;
327 |{
328 | int retval;
329 |
330 | _DIAGASSERT(nargv != NULL);
331 | _DIAGASSERT(options != NULL);
332 |
333 | if ((retval = getopt_internal(nargc, nargv, options)) == -2) {
334 |         ++optind;
335 |         /*
336 |          * We found an option (--), so if we skipped non-options,
337 |          * we have to permute.
338 |          */
339 |         if (nonopt_end != -1) {
340 |                 permute_args(nonopt_start, nonopt_end, optind,
```

```
341 |                                 nargv);
342 |                 optind -= nonopt_end - nonopt_start;
343 |         }
344 |         nonopt_start = nonopt_end = -1;
345 |         retval = -1;
346 | }
347 | return retval;
348 |}
349 |#endif
350 |
351 |/*
352 | * getopt_long --
353 | *       Parse argc/argv argument vector.
354 | */
355 |int
356 |getopt_long(nargc, nargv, options, long_options, idx)
357 | int nargc;
358 | char * const *nargv;
359 | const char *options;
360 | const struct option *long_options;
361 | int *idx;
362 |{
363 | int retval;
364 |
365 | _DIAGASSERT(nargv != NULL);
366 | _DIAGASSERT(options != NULL);
367 | _DIAGASSERT(long_options != NULL);
368 | /* idx may be NULL */
369 |
370 | if ((retval = getopt_internal(nargc, nargv, options)) == -2) {
371 |         char *current_argv, *has_equal;
372 |         size_t current_argv_len;
373 |         int i, match;
374 |
375 |         current_argv = place;
376 |         match = -1;
377 |
378 |         optind++;
379 |         place = EMSG;
380 |
381 |         if (*current_argv == '\0') {            /* found "--" */
382 |                 /*
383 |                  * We found an option (--), so if we skipped
384 |                  * non-options, we have to permute.
385 |                  */
386 |                 if (nonopt_end != -1) {
387 |                         permute_args(nonopt_start, nonopt_end,
388 |                             optind, nargv);
389 |                         optind -= nonopt_end - nonopt_start;
390 |                 }
```

```
391 |                 nonopt_start = nonopt_end = -1;
392 |                 return -1;
393 |         }
394 |         if ((has_equal = strchr(current_argv, '=')) != NULL) {
395 |                 /* argument found (--option=arg) */
396 |                 current_argv_len = has_equal - current_argv;
397 |                 has_equal++;
398 |         } else
399 |                 current_argv_len = strlen(current_argv);
400 |
401 |         for (i = 0; long_options[i].name; i++) {
402 |                 /* find matching long option */
403 |                 if (strncmp(current_argv, long_options[i].name,
404 |                     current_argv_len))
405 |                         continue;
406 |
407 |                 if (strlen(long_options[i].name) ==
408 |                     (unsigned)current_argv_len) {
409 |                         /* exact match */
410 |                         match = i;
411 |                         break;
412 |                 }
413 |                 if (match == -1)                /* partial match */
414 |                         match = i;
415 |                 else {
416 |                         /* ambiguous abbreviation */
417 |                         if (PRINT_ERROR)
418 |                                 warnx(ambig, (int)current_argv_len,
419 |                                      current_argv);
420 |                         optopt = 0;
421 |                         return BADCH;
422 |                 }
423 |         }
424 |         if (match != -1) {                      /* option found */
425 |                 if (long_options[match].has_arg == no_argument
426 |                     && has_equal) {
427 |                         if (PRINT_ERROR)
428 |                                 warnx(noarg, (int)current_argv_len,
429 |                                      current_argv);
430 |                         /*
431 |                          * XXX: GNU sets optopt to val regardless of
432 |                          * flag
433 |                          */
434 |                         if (long_options[match].flag == NULL)
435 |                                 optopt = long_options[match].val;
436 |                         else
437 |                                 optopt = 0;
438 |                         return BADARG;
439 |                 }
440 |                 if (long_options[match].has_arg == required_argument ||
```

```
441 |                     long_options[match].has_arg == optional_argument) {
442 |                         if (has_equal)
443 |                                 optarg = has_equal;
444 |                         else if (long_options[match].has_arg ==
445 |                             required_argument) {
446 |                                 /*
447 |                                  * optional argument doesn't use
448 |                                  * next nargv
449 |                                  */
450 |                                 optarg = nargv[optind++];
451 |                         }
452 |                 }
453 |                 if ((long_options[match].has_arg == required_argument)
454 |                     && (optarg == NULL)) {
455 |                         /*
456 |                          * Missing argument; leading ':'
457 |                          * indicates no error should be generated
458 |                          */
459 |                         if (PRINT_ERROR)
460 |                                 warnx(recargstring, current_argv);
461 |                         /*
462 |                          * XXX: GNU sets optopt to val regardless
463 |                          * of flag
464 |                          */
465 |                         if (long_options[match].flag == NULL)
466 |                                 optopt = long_options[match].val;
467 |                         else
468 |                                 optopt = 0;
469 |                         --optind;
470 |                         return BADARG;
471 |                 }
472 |         } else {                        /* unknown option */
473 |                 if (PRINT_ERROR)
474 |                         warnx(illoptstring, current_argv);
475 |                 optopt = 0;
476 |                 return BADCH;
477 |         }
478 |         if (long_options[match].flag) {
479 |                 *long_options[match].flag = long_options[match].val;
480 |                 retval = 0;
481 |         } else
482 |                 retval = long_options[match].val;
483 |         if (idx)
484 |                 *idx = match;
485 | }
486 | return retval;
487 |}
488 |#endif /* !GETOPT_LONG */
489 |
```

served by {OpenGrok on Java

Indexes created Tue May 20 18:09:42 GMT 2025